

1. **¿Por qué es más fácil eliminar un nodo intermedio en una lista doblemente ligada que en una lista simple?**

Es más fácil porque cada nodo tiene un puntero al siguiente y otro al anterior. Esto permite acceder directamente a los nodos vecinos del que se desea eliminar, de modo que solo se actualizan dos enlaces: el siguiente del nodo previo y el anterior del nodo siguiente. En una lista simple solo se puede avanzar hacia adelante, por lo que para eliminar un nodo intermedio es obligatorio buscar antes el nodo anterior, lo cual requiere más recorrido y complica la operación.

2. **¿Qué sucede si se olvida actualizar alguno de los punteros anterior o siguiente?**

Si no se actualiza uno de estos punteros, la estructura de la lista queda rota. Esto puede causar pérdida de nodos, recorridos incompletos, ciclos incorrectos o accesos inválidos a memoria. El recorrido hacia adelante o hacia atrás se vuelve inconsistente y la lista puede quedar fragmentada, generando errores durante las operaciones o comportamientos inesperados.

3. **¿En qué escenarios prácticos se usa una lista doblemente ligada?**

Se usa en situaciones donde es necesario recorrer los elementos en ambos sentidos o donde se realizan inserciones y eliminaciones constantes en posiciones intermedias. Ejemplos prácticos incluyen el historial de navegación en navegadores web, listas de reproducción que permiten ir al siguiente o al anterior, dequeues, estructuras de cache como LRU y algunos manejadores de memoria del sistema operativo.

Paso A2. Insercion

[Insercion inicio] Cuantos numeros deseas guardar al INICIO de la lista? 5

Valor 1/5: 1

Recorriendo lista hacia adelante:

1

Valor 2/5: 2

Recorriendo lista hacia adelante:

2 -> 1

Valor 3/5: 3

Recorriendo lista hacia adelante:

3 -> 2 -> 1

Valor 4/5: 4

Recorriendo lista hacia adelante:

4 -> 3 -> 2 -> 1

Valor 5/5: 5

Recorriendo lista hacia adelante:

5 -> 4 -> 3 -> 2 -> 1

[Insercion Final] Cuantos numeros deseas guardar al FINAL de la lista? 5

Valor 1/5: 0

Recorriendo lista hacia atras:

0 -> 1 -> 2 -> 3 -> 4 -> 5

Valor 2/5: 9

Recorriendo lista hacia atras:

9 -> 0 -> 1 -> 2 -> 3 -> 4 -> 5

```
Valor 3/5: 8
Recorriendo lista hacia atras:
8 -> 9 -> 0 -> 1 -> 2 -> 3 -> 4 -> 5

Valor 4/5: 7
Recorriendo lista hacia atras:
7 -> 8 -> 9 -> 0 -> 1 -> 2 -> 3 -> 4 -> 5

Valor 5/5: 6
Recorriendo lista hacia atras:
6 -> 7 -> 8 -> 9 -> 0 -> 1 -> 2 -> 3 -> 4 -> 5
-----
Paso A3. Recorrido en ambos sentidos
-----
Recorriendo lista hacia adelante:
5 -> 4 -> 3 -> 2 -> 1 -> 0 -> 9 -> 8 -> 7 -> 6
Recorriendo lista hacia atras:
6 -> 7 -> 8 -> 9 -> 0 -> 1 -> 2 -> 3 -> 4 -> 5
-----
Paso A4. Eliminacion
-----

Que valor deseas eliminar? 6
Recorriendo lista hacia adelante:
5 -> 4 -> 3 -> 2 -> 1 -> 0 -> 9 -> 8 -> 7
Recorriendo lista hacia atras:
7 -> 8 -> 9 -> 0 -> 1 -> 2 -> 3 -> 4 -> 5

Que valor deseas eliminar? 5
Recorriendo lista hacia adelante:
4 -> 3 -> 2 -> 1 -> 0 -> 9 -> 8 -> 7
Recorriendo lista hacia atras:
7 -> 8 -> 9 -> 0 -> 1 -> 2 -> 3 -> 4
```

```

Que valor deseas eliminar? 1
Recorriendo lista hacia adelante:
4 -> 3 -> 2 -> 0 -> 9 -> 8 -> 7
Recorriendo lista hacia atras:
7 -> 8 -> 9 -> 0 -> 2 -> 3 -> 4
-----
Paso A5. Liberacion de memoria
-----

[liberar] Liberando memoria de la lista...
free(00C61E08)
free(00C61DF0)
free(00C61DD8)
free(00C61E38)
free(00C61E50)
free(00C61678)
free(00C61690)
[liberar] Memoria liberada correctamente.

```

1. **¿Qué diferencia hay entre una lista doblemente ligada y una circular doblemente ligada?**

La diferencia principal es que en una lista doblemente ligada normal el primer nodo tiene su puntero anterior en NULL y el último nodo tiene su puntero siguiente en NULL, indicando claramente los extremos. En cambio, en una lista doblemente ligada circular no existen extremos: el primer nodo apunta hacia atrás al último y el último apunta hacia adelante al primero. Gracias a esto, la lista forma un ciclo continuo y permite recorridos circulares sin necesidad de comprobar valores NULL.

2. **¿Qué errores pueden causar ciclos infinitos al recorrer una lista circular?**

Los ciclos infinitos ocurren cuando las condiciones del bucle no detectan correctamente el regreso al nodo inicial. También pueden aparecer si se daña alguno de los punteros y el recorrido cae en un ciclo distinto al esperado. Este tipo de errores provoca que el programa nunca salga del recorrido, consuma CPU indefinidamente y evite que otras instrucciones se ejecuten, pudiendo incluso bloquear completamente la aplicación.

3. **¿Qué tipo de aplicaciones reales usan este tipo de listas (por ejemplo, reproductores multimedia, navegación en historial, juegos)?**

Las listas doblemente ligadas circulares se utilizan en aplicaciones que necesitan moverse repetidamente hacia adelante y hacia atrás sin llegar a un final. Algunos ejemplos son los reproductores multimedia con listas de reproducción en modo repetición, interfaces donde se navega continuamente entre elementos, menús

circulares de videojuegos, sistemas de turnos en juegos o simulaciones, y estructuras internas de editores que mantienen ciclos de operaciones o buffers circulares.

```
-----  
Paso B2. Insercion  
-----
```

```
[Insercion]    Cuantos numeros deseas guardar en la lista? 6
```

```
Valor 1/6: 1
```

```
Recorriendo lista hacia adelante:  
1
```

```
Valor 2/6: 2
```

```
Recorriendo lista hacia adelante:  
1 -> 2
```

```
Valor 3/6: 3
```

```
Recorriendo lista hacia adelante:  
1 -> 2 -> 3
```

```
Valor 4/6: 4
```

```
Recorriendo lista hacia adelante:  
1 -> 2 -> 3 -> 4
```

```
Valor 5/6: 5
```

```
Recorriendo lista hacia adelante:  
1 -> 2 -> 3 -> 4 -> 5
```

```
Valor 6/6: 6
```

```
Recorriendo lista hacia adelante:  
1 -> 2 -> 3 -> 4 -> 5 -> 6
```

```
-----  
Paso B3. Recorrido  
-----
```

```
Recorriendo lista hacia adelante:  
1 -> 2 -> 3 -> 4 -> 5 -> 6
```

Paso B4. Eliminacion

Que valor deseas eliminar? 1
Recorriendo lista hacia adelante:
2 -> 3 -> 4 -> 5 -> 6

Que valor deseas eliminar? 6
Recorriendo lista hacia adelante:
2 -> 3 -> 4 -> 5

Que valor deseas eliminar? 3
Recorriendo lista hacia adelante:
2 -> 4 -> 5